

KEJRIWAL Z-Portfolio → SAP S/4HANA 2025 Fiori

Module-wise build inventory & development-server runbook

Clean-core extension package · KSQ/KHQ · client 500

Generated 26 June 2026 · All 285 Z-transactions classified & routed
Repository: [ajaykejriwal1974-oss/fiori-apps](#) (merged to main)

Executive summary

The entire custom Z-portfolio — **285 transactions** — has been classified, routed clean-core (no SAP source modified), and for everything that warrants custom code, scaffolded with real fields from the field dictionary. This document lists the work **by SAP module** and gives a **step-by-step plan for the development server**.

Class	Count	Disposition
CUS	72	Built (masters + transactional) or reuse / DRC excluded
EXT	19	Built / wired into adaptations
STD	17	Standard Fiori app + business role
PRT	61	Output Management (gate pass built)
BI	87	Consolidated → ~31; 11 CDS queries built
UPL	28	Migration Cockpit / APIs; 2 automation classes built
RET	1	Retire

Clean core throughout: key-user / adaptation / RAP / released-API only — never a modification. Module grouping below is functional; textile logistics (packing / HU / gate pass) legitimately spans PP, MM and SD.

Part A — Deliverable register (module-wise, by code)

Every app, master and report built in this package, grouped by SAP module, each with a **delivery code** and its **complete set** of replaced Z-transactions (expanded from docs/CLASSIFICATION.md + docs/BI-CONSOLIDATION.md, not just the representative tcode). **29** apps/masters (-NN), **11** analytical reports (-RN), **118** distinct Z-tcodes covered. Type: *Adaptation* = key-user extension of a standard app; *Custom app* = freestyle SAPUI5; *Master (RAP)* = managed-RAP Fiori Elements; *Report* = analytical CDS query. † = retired duplicate folded into the consolidated query.

SD Sales & Distribution

Code	App / Report	Type	Replaces (Z-tcodes)
SD-01	Manage Sales Orders (adaptation, F1873)	Adaptation	ZVA01, ZVA01N, ZSOCLOSE, ZSOCLOSE1
SD-02	Manage Sales Contracts (adaptation)	Adaptation	ZCON_CLOSE, ZCON_CLOSE1, ZCOREL, ZCON02
SD-03	Manage Outbound Deliveries (adaptation, F0867A)	Adaptation	ZDEL, ZDELC
SD-04	Contract Batch Update	Custom app	ZBATCH_CHANGE
SD-05	Dispatch Correction	Custom app	ZDSP_CORR
SD-06	Manage Packing Details	Custom app	ZPACK01, ZPACK01N, ZPACK02, ZPACK02N, ZPACK03, ZREPACK
SD-07	Dyeing Packing	Custom app	ZPACK01D, ZPACK02D, ZPACK03D, ZREPACKD
SD-08	Transport Code Master	Master (RAP)	ZTRANS
SD-09	Truck Master	Master (RAP)	ZTRUCK
SD-10	Export Details Master	Master (RAP)	ZMBR2
SD-R1	Packing / Dispatch Register (17 → 1)	Report	ZPACKLIST, ZPACKLIST01, ZPACKLISTN, ZPLIST01, ZPLIST01A, ZPLIST01T, ZPLIST02, ZPLIST02A, ZPLIST02N†, ZPLIST02T, ZPLIST03, ZPLIST03A, ZPLIST03N†, ZPLIST03T, ZPLISTD, ZPLISTDA, ZPLISTDT
SD-R2	Dispatch Register	Report	ZPWDIS, ZDISPATCH, ZPDESP, ZSCAND
SD-R3	Export / Incentive Register	Report	ZGCUDB, ZBRC, ZEXP
SD-R4	Pending Contract Register	Report	ZPCON, ZPCOND, ZPCONS, ZPCON_CP†

PP Production Planning

Code	App / Report	Type	Replaces (Z-tcodes)
PP-01	Confirm Production Operation (adaptation, F3069)	Adaptation	ZC011N, ZC011A
PP-02	Palletization	Custom app	ZPALLET, ZPALLET1, ZPAL_BOX, ZSOL_ASRS
PP-03	Post Packing & GR	Custom app	ZPOST01
PP-04	Batch Status	Custom app	ZBATCHD, ZBATCH_CLS
PP-05	Recipe Master	Master (RAP)	ZRECP01, ZRECP02, ZRECP03
PP-06	Job Master	Master (RAP)	ZJOB01, ZJOB01N, ZJOB02, ZJOB02N, ZJOB03, ZJOB03N
PP-07	Schedule Master	Master (RAP)	ZSCH01, ZSCH01N, ZSCH02, ZSCH02N, ZSCH03, ZSCH03N
PP-08	Merge Details Master	Master (RAP)	ZMERGE
PP-09	Checked / Packed-By Master	Master (RAP)	ZPCBY
PP-10	Packing Material Master	Master (RAP)	ZPACK_MAST
PP-11	Shade Master (Dope Dyeing)	Master (RAP)	ZDD_SHADE
PP-R1	Packed-Stock Analysis (8 → 1)	Report	ZBOXSTOCK, ZGSTOCK, ZPRP1, ZSSTOCK, ZDSTOCK, ZSTOCK, ZPRP, ZPRPSZ
PP-R2	WIP Batch	Report	ZBATCH_WIP
PP-R3	Job Card Report	Report	ZJOBREPTN, ZJOBREPORT†
PP-R4	Recipe Master Report	Report	ZRECPM
PP-R5	Merge Analysis	Report	new analytics over ZPP_MERGE (Merge master; no standalone legacy report)

MM Materials Management & Handling Units

Code	App / Report	Type	Replaces (Z-tcodes)
MM-01	Post Goods Movement (HU / Box)	Custom app	ZBOX_MOVE
MM-02	Inbound Delivery HUs	Custom app	ZHUINB
MM-03	HU Unpack	Custom app	ZHUPK
MM-04	MTOS Process	Custom app	ZMTOS, ZHUINV
MM-05	Gate Pass	Master (RAP)	ZGPS01, ZGPS02, ZGPS03, ZGPSI1, ZGPSI2, ZGPSI3
MM-R1	HU Inventory Analysis	Report	ZHUM0, ZHUREC, ZHUINV_CLS

QM Quality Management

Code	App / Report	Type	Replaces (Z-tcodes)
QM-01	Record Inspection Results (Mass)	Custom app	ZQA32

FI Financial Accounting & Compliance

Code	App / Report	Type	Replaces (Z-tcodes)
FI-01	C-Form Allocation Master	Master (RAP)	ZCFORM1, ZFORM, ZFORMS, ZPCFORM
FI-02	Digital Signature Master	Master (RAP)	ZDIGI
FI-R1	GST Tax Register	Report	ZGST, ZGST1, ZGST2, ZGSTCR

Part B — Build inventory by module

SD Sales & Distribution

Sales orders, contracts, deliveries, packing, dispatch and the export/sales registers.

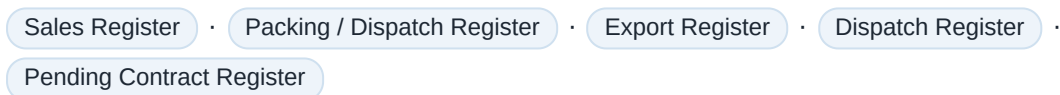
Apps built

- **Manage Sales Orders (F1873) — adaptation** — textile custom fields + Close Order
- **Manage Sales Contracts — adaptation** — Close / Complete / Release / Update-Rate
- **Manage Outbound Deliveries (F0867A) — adaptation** — delivery challan via Output Mgmt
- **Contract Batch Update** — mass batch assignment across contract items
- **Dispatch Correction** — re-assign dispatch boxes (SO/item/status)
- **Manage Packing Details** — pack / repack (ZPACK01..03N, ZREPACK)
- **Dyeing Packing** — cone/carton/pallet HU structure (ZPACK*D)

Backend (RAP / classes)

- **sales-doc-status-rap** — merged VBAK service: 6 lifecycle actions (contracts + orders)
- **contract-batch-rap** — BAPI_SD_SALESDOCUMENT_CHANGE (batch)
- **dispatch-correction-rap** — ZSOL_HUDISPATCH ⇄ ZPP_PACK
- **packing-detail-rap / packing-hu-rap** — BAPI_HU_PACK family
- **transport-code-master / truck-master / export-detail-master** — managed RAP over ZTRANS / ZTB_TRUCK_MSTR / ZEXP

Analytics



Reuse / standard

Sales-register variants → 1 query; ZVA01/N custom fields via key-user adaptation; reuse ZSOL_SALCONT_CDS / ZSOL_F4* value helps.

Representative Z-tcodes

ZVA01/N, ZSOCL0SE(1), ZCON_CLOSE/CLOSE1/COREL/CON02, ZDEL/ZDELCL, ZBATCH_CHANGE, ZDSP_CORR, ZPLIST*/ZPACK*, ZSALES*, ZSOREG, ZGCUDB/ZBRC/ZEXP

PP Production Planning

Dyeing production entry, batches, job cards, schedules, recipes, merge, palletization and the production/stock analytics.

Apps built

- **Confirm Production Operation (F3069) — adaptation** — dyeing production entry (ZCO11N/A)
- **Palletization** — pack boxes on pallet (ZPALLET/1, ZPAL_BOX)
- **Post Packing & GR** — post packing + goods receipt (ZPOST01)
- **Batch Status** — close / delete batch (ZBATCHD, ZBATCH_CLS)

Backend (RAP / classes)

- **recipe / job / schedule / merge / checked-by / packing-material masters** — managed RAP over the real ZPP_* tables
- **shade-master-rap** — reference managed CBO (ZDD_SHADE)
- **gate-pass-rap** — managed RAP composition ZGP_HDR → ZGP_ITEM
- **batch-status / palletization / post-packing-gr** — unmanaged RAP action services

Analytics

Packed-Stock Analysis (8 reports → 1) · WIP Batch · Job Card · Recipe Analysis · Merge Analysis

Reuse / standard

Create/Change/Display master variants collapse into one Fiori Elements app each; recipe/job-card PRINT becomes an Output-Management form on the master.

Representative Z-tcodes

ZCO11N/A, ZRECP*, ZJOB*, ZSCH*, ZBATCH* (master/del/close/WIP), ZMERGE, ZPALLET*, ZPOST01, ZREPACK, ZPRP*, ZLABEL, ZPCBY

MM Materials Management & Handling Units

Handling-unit handling, goods movement, gate pass, purchasing automation, MTO → MTS and stock.

Apps built

- **Post Goods Movement (HU / Box)** — box/HU-wise movement (ZBOX_MOVE)
- **Inbound Delivery HUs** — post inbound GR (ZHUINB)
- **HU Unpack** — RM unpack (ZHUPK)
- **MTOS Process** — MTO → MTS transfer + phys-inv doc (ZMTOS+ZHUINV, merged)

Backend (RAP / classes)

- **goods-movement-hu / hu-inbound / hu-unpack / mtos-process** — unmanaged RAP over standard HU (VEKP/VEPO)
- **hu-shared** — shared ZI_HU_ItemBase / ZI_HU_HeaderBase (used by 7 services)
- **gate-pass-rap** — the custom gate-pass object (also serves MM inward/outward)
- **po-automation / obd-automation** — ABAP job classes: PO & delivery automation
- **minmax-master-rap** — reuse standard MRP on MARC (stub)

Analytics

HU Inventory Analysis (ZHUMO/ZHUREC/ZHUINV_CLS → 1)

Reuse / standard

Generic stock (ZMB5B/ZCMM001/ZBSTOCK) → standard inventory apps; ZAUTOPO* (9) → one PO-automation class; reuse ZSOL_INBOUND_HU / ZSOL_POST_GOODS_MOVEMENTS / ZSOL_PALLETIZATION.

Representative Z-tcodes

ZBOX_MOVE, ZHUINB, ZHUPK, ZHUINV, ZMTOS, ZGP*/ZGATE* (gate pass), ZAUTOPO*, ZME49, ZMM60, ZPR, ZRFQ, ZWMT0_UPLD, ZMB5B/ZBSTOCK

QM Quality Management

Mass inspection-result recording and inspection-lot handling.

Apps built

- **Record Inspection Results (Mass)** — multi-lot result entry (ZQA32) — vs F2655 one-lot

Backend (RAP / classes)

- **qm-mass-results-rap** — reads standard QM (QAMV/QALS/QAPO/QAMR); posts via QM result API

Reuse / standard

ZINSLOT may map to standard Create Inspection Lot; ZQAR (cancel lot) and ZQ_FORM (cert) → standard + Output Management.

Representative Z-tcodes

ZQA32, ZINSLOT, ZQAR, ZQ_FORM

FI Financial Accounting & Compliance

Line-item display, ageing, statements, GST/TDS, banking, assets, C-Form, Bill of Exchange and posting automation.

(no custom Fiori app — see masters & reuse) FI is mostly standard apps + automation

Backend (RAP / classes)

- **cform-master-rap** — C-Form allocation (ZCFORM1) — pending ↔ received
- **digital-signature-master-rap** — ZTDIGI_SIGN configuration
- **bill-of-exchange-std** — reuse standard FI Bill of Exchange (stub)
- **po-automation (FI posting variants)** — F-02 BDC clones → Journal Entry API

Analytics

Ageing · Account Statement · Interest · TDS · GST Tax Register · Commission · Credit/Debit Note

Reuse / standard

Line-item apps (FBL3N/5N), cheque (Check Mgmt), Bill of Exchange (F-36/F-33/FBW*), GST/excise (DRC) all reuse STANDARD; FI uploads → Migration Cockpit / Journal Entry API; all BDC retired.

Representative Z-tcodes

ZFBL3N/5N, ZF02/ZFIEX/ZFIVT, ZFI* (ageing/stmt/interest/TDS), ZGST*, ZCHQ, ZABMA, ZCFORM*, ZBOE, ZCRDR*, ZBANK

CROSS Cross-application / Platform / Compliance

Document & Reporting Compliance (e-invoice/e-way), Output Management, data migration and platform utilities.

(platform / config — not Fiori tiles)

Backend (RAP / classes)

- **DRC (India e-invoice / e-way)** — ZEINV*/ZEWAY*/ZEXN → standard SAP DRC (excluded from build)
- **Output Management (PRT layer, 61)** — forms/labels/stickers/gate-pass print → BRF+ + Adobe templates
- **Data migration (UPL layer, 28)** — uploads → Migration Cockpit; automation → APIs / Application Jobs
- **digital signature / PDF / task list / audit log** — platform utilities

Reuse / standard

DRC replaces the entire e-invoice/e-way family; PRT → Output Management; UPL → Migration Cockpit + public APIs (BDC retired).

Representative Z-tcodes

ZEINV*, ZEWAY*, ZEXN, ZPDF, ZDIGI, ZTASKLIST, ZAUDIT_LOG, + 61 PRT + 28 UPL programs

Part C — From GitHub to the server (import & transport)

Getting the source **out of GitHub and into SAP** is **not** an SAP transport — a transport (STMS) only moves objects *within* the landscape. Git → SAP is a separate **import** step, with different tools per layer. Three stages: **import** to DEV (abapGit for backend, fiori deploy for UI) → **bindings + launchpad** → **STMS** up to KSQ/PROD.

```
GitHub repo --(import)--> SAP DEV --(STMS transport)--> KSQ --> PROD
      abapGit (backend) + fiori deploy (UI)          TR-01 ... TR-FLP
```

1 Stage 1A — Backend ABAP → DEV via abapGit

The backend is serialized in ADT/abapGit format (.ddls / .bdef / .clas / .srvd), so abapGit recreates it as real repository objects. In DEV: open abapGit (ADT plug-in or ZABAPGIT_STANDALONE); create the ZKEJRIWAL_BE_* Z packages first (never \$TMP); clone/link the repo and pull; then activate in FU#2 dependency order. Objects activating under a Z package collect onto a workbench transport. One package ↔ one repo/folder, so either point a per-feature abapGit repo at each backend/<feature> folder, or import an offline ZIP per package.

Tooling: abapGit · TRANSPORT-PLAN.md §1 · ADT-ACTIVATION-CHECKLIST FU#2

2 Stage 1B — UI apps → DEV (FES) via fiori deploy

The SAPUI5 apps deploy as BSP applications, not abapGit objects. Fill the REPLACE_WITH_* placeholders, then npm install && npm run deploy (fiori deploy) pushes each app's BSP to the embedded Front-End Server and attaches it to a workbench TR — set packageName off \$TMP to a real ZKEJRIWAL_UI_* package first. The four adaptation projects deploy the same way as app variants.

Tooling: PUBLISHING.md · WIRING-CHECKLIST.md

3 Air-gapped DEV (no internet to GitHub)

The SAP box needs no internet. Backend: download the repo ZIP and use abapGit Import → from ZIP per package. UI: run npm run deploy from a workstation/CI box that reaches both npm and the FES. If DEV does clone GitHub over HTTPS, first import the GitHub TLS chain into STRUST or the clone fails on certificate verification.

Tooling: abapGit offline · STRUST

4 Stage 2 — Service bindings + launchpad (per system)

Create + activate/publish each OData V4 service binding (FU#1); fill the REPLACE_WITH_*_SERVICE manifest tokens and controller SERVICE_NS. Build launchpad content (catalogs, target mappings, tiles, spaces/pages, PFCG roles) as TR-FLP. Bindings transport as objects but must be re-activated in each target system.

Tooling: ADT-ACTIVATION-CHECKLIST FU#1 · PUBLISHING.md

5 Stage 3 — DEV → KSQ → PROD via STMS

Now it is ordinary STMS. Group objects into the feature transports (TRANSPORT-PLAN.md §3) and release in order: TR-01 Shade foundation → config TRs → TR-02..TR-09 features → TR-FLP launchpad → roles. On the embedded FES the backend + UI ride the same route. Import the same ordered set into KSQ, validate, sign off, then PROD.

Tooling: TRANSPORT-PLAN.md §3–4 · GO-LIVE-CHECKLIST.md

Full detail: docs/GITHUB-TO-SERVER.md. The SAP server never "pulls a transport from GitHub" — a developer or CI imports the source, then the activated objects are transported.

Part D — Next steps on the development server (in order)

Once the source is in DEV (Part C, Stage 1), this is the activation/wiring sequence. Earlier steps unblock later ones; masters need the least wiring, the transactional BAPI bodies the most. References point to the repo docs.

1 Prerequisites & baseline

Confirm ADT access to the DEV system feeding KSQ/KHQ (client 500), the embedded Front-End Server is active, and the standard base apps (F1873, F3069, F0867A, Manage Sales Contracts, F2655, MIGO/HU) are activated via Basis Rapid Activation. Import the repo objects into a development package + transport request (see TRANSPORT-PLAN.md).

Reference: ACTIVATION.md, TRANSPORT-PLAN.md

2 Activate the managed-RAP masters first (lowest effort)

For each master (Recipe, Job, Schedule, Transport, Truck, Merge, Checked-by, Packing-Material, Export, Digital-Signature, C-Form, Shade, Gate Pass): confirm the underlying legacy table exists (all bind EXISTING tables; only ZDD_SHADE may need its table created), then activate the CDS interface → projection → behavior definition → behavior class → metadata extension → service definition. Create the OData V4 service binding (e.g. ZUI_RECIPES_O4) and publish it.

Reference: backend/*-master-rap, backend/gate-pass-rap

3 Generate & test the master Fiori Elements apps

Each master's List Report / Object Page is generated from its .ddlx via the service binding — no UI code. Launch from the binding preview, verify the field list against the live data, then wire the Gate-Pass number range (ZGPASS_NUM) in setHeaderDefaults and reuse ZSOL_F4* value helps.

Reference: WIRING-CHECKLIST.md §1, §5

4 Wire the adaptation projects (SD + PP)

For the 4 adaptation projects, fill the base-app identifiers from the activated standard apps: REPLACE_WITH_BASE_APP_COMPONENT_ID / BSP_NAME / FIORI_ID / OBJECT_PAGE_VIEW_ID / CONTROLLER_NAME / EXTENSION_POINT_NAME. Deploy the app-variant; confirm the custom section/column/controller extension renders on the standard app.

Reference: apps/manage-sales-orders-ext, manage-sales-contracts-ext, manage-outbound-deliveries-ext, confirm-production-operation-ext

5 Implement the unmanaged transactional services (the BAPI bodies)

For each transactional service, replace the TODO in the behavior class with the real BAPI call: sales-doc-status → BAPI_SD_SALESDOCUMENT_CHANGE; mtos-process → BAPI_GOODSMVT_CREATE + BAPI_MATPHYSINV_CREATE_MULT; packing/HU → BAPI_HU_PACK/_REPACK/_UNPACK/_CREATE; goods-movement → BAPI_GOODSMVT_CREATE; hu-inbound → BAPI_INB_DELIVERY_CONFIRM_DEC; batch-status → BAPI_BATCH_CHANGE; dispatch-correction → ZSOL_DISPATCH_CORRECTION; qm-mass-results → QM result API. Add BAPI_TRANSACTION_COMMIT and map the BAPI return into reported/failed. Resolve the 5 VERIFY markers (release-specific filters).

Reference: WIRING-CHECKLIST.md §3, REUSE-EXISTING.md

6 Create the service bindings & wire the freestyle apps

Create the OData V4 binding for each transactional service (ZUI_*_O4) and put the name into each app's manifest.json (REPLACE_WITH_*_SERVICE) and the two adaptation controllers (REPLACE_WITH_SALESDOC_STATUS_SERVICE). Then wire the front-end action invocation (oOperation.invoke()) with the parameter structure) in the 12 worklist/controller TODOs; set REPLACE_WITH_ABAP_SYSTEM_URL in each ui5.yaml for local test.

Reference: WIRING-CHECKLIST.md §1, §4

7 Build the analytical queries (BI)

Activate the 11 CDS cube/query pairs in backend/analytics. Switch @AccessControl.authorizationCheck from #NOT_REQUIRED to #CHECK with a DCL. Expose each ZC_*Query via the Query Browser or an Analytical List Page; add the secondary-object joins noted (billing tax amounts, contract value, dispatch weight). Map the retired report users to the new query + role.

Reference: backend/analytics, BI-CONSOLIDATION.md

8 Register the automation jobs (UPL)

Activate ZCL_PO_AUTOMATION and ZCL_OBD_AUTOMATION; complete the API calls and the Application-Job parameter definitions; register each as an Application Job catalog entry and schedule it (VKORG / WERKS parameters). Build the remaining UPL items as Migration Cockpit objects / standard API clients — retire all BDC.

Reference: backend/po-automation, backend/obd-automation, UPL-MIGRATION.md

9 Output Management & statutory (PRT + DRC)

Re-platform the 61 PRT programs onto Output Management: one BRF+ determination + Adobe form template per output object (billing, delivery, gate pass, labels). Configure standard SAP DRC for India e-invoice / e-way bill (the excluded ZEINV* family). Retire the pre-GST 201-series and RG1 forms.

Reference: PRT-OUTPUT-MANAGEMENT.md

10 Retire the STD & reuse items; assign roles

Assign the 17 STD standard apps + the reuse-standard items (Min/Max on MARC, Bill of Exchange, generic stock, ageing/statements) to the relevant SAP_BR_* business roles; publish tiles in the users' Spaces/Pages; retire the corresponding Z transactions after a parallel run.

Reference: STD-RETIREMENT.md, REUSE-EXISTING.md

11 Validate, transport, go-live

Run ATC on every activated object, execute the per-app GO-LIVE-CHECKLIST, then transport DEV → KSQ → PROD per the TRANSPORT-PLAN. The repo's structural CI (ci/validate.py) gates JSON/XML/CDS/ABAP structure on each change.

Reference: GO-LIVE-CHECKLIST.md, TRANSPORT-PLAN.md, ci/validate.py